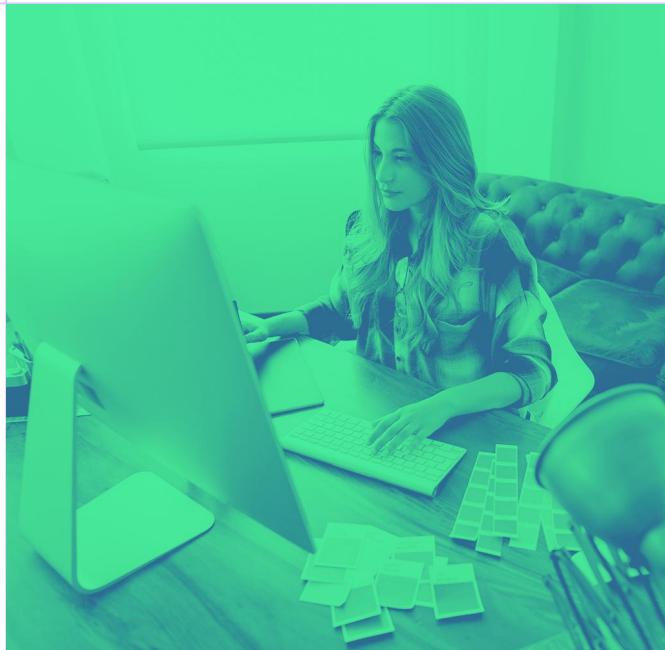




Sécurité WEB

STIENNE Dylan





Sécurisation WEB ?

Qui sera le plus chiant ? Gain / effort

- Sécurisation Front-End
- Sécurisation des APIs
- Sécurisation des Webhooks
- Sécurisation des données
- Sécurisation des fichiers
- Les middlewares
- La gestion de droits



Sécurisation Front-End



Protection contre le XSS

Faible de sécurité où un attaquant injecte du code JavaScript malveillant dans une page web, souvent via des champs de formulaire.

Solutions :

- Échapper les données dynamiques (HTML, attributs, scripts)
- Utiliser `innerText` au lieu de `innerHTML`
- Utiliser des bibliothèques de sanitisation comme `DOMPurify`
- Éviter d'exécuter du contenu dynamique (`eval`, `innerHTML`, etc.)



Headers de sécurité

À mettre dans les headers du server

- Content-Security-Policy : restreint les scripts externes (Ex. GTM)
- X-Frame-Options : empêche l'iframe (clickjacking)
- X-Content-Type-Options : empêche les interprétations de type (html != .exe)
- Strict-Transport-Security (HSTS) : force l'utilisation de HTTPS



Cookies sécurisés

Ne fonctionne qu'avec un backend stateful (sessions serveur)

- HttpOnly : les cookies ne sont pas accessibles via le JS
- Secure : les cookies ne sont transmis uniquement en HTTPS
- SameSite=Strict : empêche les attaques CSRF



Gestion des erreurs / logs

- Ne jamais afficher de stack trace ou d'informations techniques
- Utiliser un système de logs interne pour les erreurs
- En prod : désactiver les messages d'erreurs détaillés
- Personnaliser les pages d'erreur (404, 500)



Sécurisation des APIs



API Key & Basic Auth

- API Key : simple mais pas super sécurisé (vol, hash, régénération)
- Basic Auth : encodage base64, à éviter sauf usage interne
- OAuth: les données sont dans des identities providers
- Toujours passer ces infos via HTTPS
- ⚠ Ne pas exposer les clés dans le frontend



JWT

- Structure : header.payload.signature
- Inclure une expiration (`exp`) courte
- Ne jamais inclure de données sensibles dans le payload
- Stocker dans cookie HttpOnly ou mémoire volatile (si stateless)



HTTPS

- Chiffrement des échanges client-serveur
- Utilisation de SSL/TLS (certificat valide)
- Redirection automatique de HTTP → HTTPS
- ⚠ Requis pour sécuriser cookies, tokens, auth



CORS

Ne s'applique qu'avec les navigateurs

- Limiter les origines autorisées (`Access-Control-Allow-Origin`)
- Autoriser uniquement les méthodes nécessaires (GET, POST...)
- Utiliser `Access-Control-Allow-Credentials` si cookie
- ⚠ Ne jamais autoriser `\*` pour les requêtes authentifiées



Rate Limiting

- Limiter le nombre de requêtes par IP ou utilisateur
- Protection contre brute force ou DDoS légers
- Utiliser des outils comme `express-rate-limit`, Cloudflare



Les inputs

- La plus grande porte d'entrée
- Très facile à casser
- Il faut tout formater / vérifier (via zod par exemple)
- 3 types d'inputs
 - Body
 - Path params
 - Query params



Sécurisation des données



Hashage

- Utiliser `bcrypt`, `argon2`, `pbkdf2` (jamais MD5 ou SHA1)
- Toujours ajouter un `salt` unique par mot de passe
- Configurer un facteur de coût élevé (ex: 10-12 pour bcrypt)
- Comparer les hashes via fonctions sécurisées



Chiffrement

- Utiliser AES-256, pour les données sensibles
- Stocker la clé de chiffrement hors du code source (.env)
- Appliquer le chiffrement aux données critiques (CB, tokens)
- Ne pas chiffrer inutilement les données non sensibles



Logs sensibles

- Ne jamais logger les mots de passe ou tokens
- Filtrer automatiquement les données sensibles (Ex. CI/CD vercel ou gitlab)
- Utiliser un outil de log centralisé sécurisé



Droits de base de données

- Ne pas utiliser l'utilisateur root
- Créer des utilisateurs avec droits limités (RLS)
- Séparer les accès lecture/écriture si possible
- Activer l'audit des connexions sensibles



Sécurisation des Webhooks



Pourquoi ?

- Toute source externe peut appeler l'endpoint
- Risque de spoofing ou envoi malveillant
- Besoin de valider l'intégrité et l'origine
- Toujours vérifier la signature



Signature HMAC

- Signature avec secret partagé (HMAC SHA256)
- Recalculer la signature côté serveur
- Comparer avec ``timingSafeEqual`` pour éviter les attaques temps
- Ne jamais parser le JSON avant vérification



Protection replay

- Inclure un timestamp dans la requête
- Rejeter les requêtes trop anciennes (> 5 min)
- Utiliser un `nonce` pour éviter la rediffusion
- ⚠️ Attaques possibles si pas de TTL



Sécurisation des fichiers



Uploads

- Vérifier : extension, type MIME, taille
- Refuser les types exécutables (.php, .exe...)
- Limiter la taille maximale de l'upload
- Utiliser un scanner antivirus (optionnel)



Accès & renommage

- Renommer les fichiers (UUID, hash...)
- Ne jamais utiliser le nom d'origine
- Placer les fichiers dans un dossier non exposé
- Restreindre les chemins d'accès aux fichiers



Permissions serveur

- chmod et chown : limiter les droits
- Empêcher l'exécution des fichiers uploadés
- Isoler les dossiers de stockage
- Journaliser les accès aux fichiers sensibles



Stockage externe

- Utiliser des URLs signées avec expiration (ex: S3)
- Ne jamais exposer des fichiers privés en public
- Séparer les fichiers publics/privés
- Restreindre l'accès via des permissions ACL



Les middlewares



Définition

- Fonction exécutée avant/après une route
- Permet d'ajouter des règles de sécurité globales
- Exemples : auth, log, IPs, hmac
- Supportés par la majorité des frameworks modernes
- En front et en back



Authentification

- Valide le token (JWT, session...)
- Ajoute les infos utilisateur au contexte
- Redirige si non authentifié
- ⚠ Ne pas exposer d'info sensible en cas d'échec



Autorisation

- Vérifie les droits d'accès selon rôle/ACL
- Bloque ou redirige les utilisateurs non autorisés
- Exemple : middleware adminOnly
- Combinaison possible avec RBAC/ABAC



Sécurité générale

- Rate limiter, logger, contrôle IP
- Middleware de sanitation des données entrantes
- CSRF protection sur formulaires sensibles



Gestion de droits



RBAC (Role-Based Access Control)

- Rôles définis : admin, user, viewer...
- Chaque rôle a un ensemble d'actions autorisées
- Simple à implémenter et auditer
- Bon pour les structures fixes



ABAC (Attribute-Based Access Control)

- Droits basés sur les attributs : created_by, group, heure, statut
- Très flexible mais plus complexe
- Permet des règles dynamiques avancées
- Utilisable en complément du RBAC



ACL (Access Control List)

- Liste explicite des accès autorisés par ressource/utilisateur
- Très précis : permet des règles fines
- Peut devenir complexe à gérer à grande échelle
- Bon pour des ressources granulaires (fichiers, objets)
- Ne pas utiliser pour des users



Le projet



Partie 1 - La base

- Mettre en place une base d'api vide (via le framework de votre choix)
- Créer un endpoint GET /health qui retourne {test: "hello world"}
 - Tester avec Postman ou autre
- Mettre en place une base de donnée SUPABASE
 - Créer une table "users" (ne pas utiliser l'auth supabase)
- Créer un endpoint POST /register pour s'inscrire
 - Nom
 - Email
 - Mot de passe



Partie 2 - L'auth

- Créer un endpoint POST /login pour recevoir un token JWT
 - Le token doit être valable 1 heure
- Créer un endpoint GET /my-user pour recevoir toutes nos infos
 - Il faut passer le token JWT dans le header Authorization
 - Il faut créer un Authorizer (qui s'occupe d'identifier le user via le token)
- Créer un endpoint GET /users pour recevoir la liste de tous les users
 - On doit être authentifié
- Si l'utilisateur change son mot de passe, alors son JWT s'expire de suite
- Une tentative de connexion possible toutes les 5 secondes



Partie 3 - Les rôles

- Créer une table “roles” et sa relation sur la table “users”
 - Nom
 - can_post_login
 - can_get_my_user
 - can_get_users
- Créer un rôle ADMIN (toutes les permissions)
- Créer un rôle USER (can_get_users == false)
- Créer un rôle BAN (aucune permission)
- Créer un user ADMIN, un BAN et mettre les le rôle USER par défaut
- Créer un middleware s'occupant de vérifier les permissions d'utilisations des différents endpoints (on peut l'inclure dans le middleware d'authentification)